

Tiempo polinomial y no polinomial

Comentario: ningún ejercicio reviste mayor dificultad.

Ejercicio 1. Responder breve y claramente los siguientes incisos:

a. ¿Por qué la complejidad temporal sólo trata los lenguajes recursivos?

Porque para medir el tiempo necesitás un final; no tiene sentido intentar medir o comparar el tiempo de una máquina que loopea infinitamente.

La complejidad exige una cota finita de pasos ($T(n)$), algo imposible en lenguajes que no garantizan frenar ante un "No".

Por eso P, NP y EXP viven dentro de los recursivos (R): si el problema no es decidable, no tiene sentido medir tiempo.

b. Probar que $n^3 = O(2^n)$.

Ayuda: hay que encontrar un número natural n_0 y una constante $c > 0$ tales que $n^3 \leq c \cdot 2^n$ para todo $n \geq n_0$.

Definición formal de BigO:

Formalmente se define:

$$O(g(x)) = \left\{ f(x) : \text{existen } x_0, c > 0 \text{ tales que } \forall x \geq x_0 > 0 : 0 \leq |f(x)| \leq c|g(x)| \right\}$$

Una función $f(x)$ pertenece a $O(g(x))$ cuando existe una constante positiva c tal que a partir de un valor x_0 , $f(x)$ no sobrepasa a $cg(x)$. Quiere decir que la función f es inferior a g a partir de un valor dado salvo por un factor constante.

Tenemos que encontrar una constante C y un número n_0 , tales que, a partir de cierto punto, la exponencial le gane por goleada a la cúbica.

Voy a fijar $C = 1$, y comparar las otras 2.

n	n^3 (Cúbica)	2^n (Exponencial)	¿Se cumple $n^3 \leq 2^n$?
1	1	2	SÍ ($1 \leq 2$)
2	8	4	NO ($8 > 4$)
5	125	32	NO ($125 > 32$)
9	729	512	NO ($729 > 512$)
10	1000	1024	SÍ ($1000 \leq 1024$)
11	1331	2048	SÍ ($1331 \leq 2048$)

Entonces, sabemos que para $n_0 = 10$ y $c = 1$, la desigualdad se cumple.

Para cualquier $n > 10$ la brecha se hace cada vez más grande.

Dado que encontramos un par de constantes n_0 y c tales que satisfacen $n^3 \leq c \cdot 2^n$, para todo $n \geq 10$, queda demostrado que $n^3 = O(2^n)$

c. Probar que si $T1(n) = O(T2(n))$, entonces $TIME(T1(n)) \subseteq TIME(T2(n))$.

Ayuda: hay que probar que si un lenguaje L está en $TIME(T1(n))$, también está en $TIME(T2(n))$ - recurrir directamente a la definición de orden O de una función T y de clase $TIME(T(n))$ -.

Contexto:

- **Que son $T1$ y $T2$?** La T viene de tiempo. $T(n)$ es una función temporal, siendo n el tamaño de las entradas, que se usa para especificar el crecimiento temporal de un algoritmo que ejecuta una MT, en relación al crecimiento de las entradas.
- **Qué quiere decir $T1(n) = O(T2(n))$?** Que $T2$ es un techo para $T1$, nunca lo va a pasar.
- **Y qué es $TIME(T(n))$?** Es una clase que agrupa a todos los lenguajes del universo que pueden ser decididos en tiempo $O(T(n))$

Nos piden probar que, si el tiempo $T1$ entra adentro de la bolsa de $T2$ ($T1 = O(T2)$), entonces cualquier lenguaje que se resuelva en tiempo $T1$ también se puede resolver en tiempo $T2$.

*Me falta desarrollar la prueba

d. ¿Cuándo un lenguaje pertenece a P, a NP y a EXP? ¿Por qué si un lenguaje pertenece a P también pertenece a NP y a EXP?

- **P:** Un lenguaje pertenece a la clase P si existe una MT M que lo **decide** en tiempo polinomial.
- **NP:** Un lenguaje pertenece a la clase NP si la pertenencia de una cadena al mismo se puede **verificar** en tiempo polinomial, con la ayuda de otra cadena complementaria (solución a chequear) conocida como certificado o prueba. De manera equivalente, un lenguaje pertenece a esta clase si existe una máquina de Turing no determinística que lo decide en tiempo polinomial.
- **EXP:** Un lenguaje pertenece a la clase EXP si es decidible en tiempo exponencial
- **¿Por qué si un lenguaje pertenece a P también pertenece a NP?**
 - Un problema está en NP si existe un algoritmo que verifica una solución en tiempo polinomial. Si un problema ya está en P, significa que existe un algoritmo que encuentra la solución en tiempo polinomial. Entonces si vos ya sabés cómo encontrar la solución rápido (P), entonces verificarla es trivial: podés simplemente ignorar el certificado que te den y resolvés el problema por tu cuenta.
- **¿Por qué si un lenguaje pertenece a P también pertenece a EXP?**
 - Si un algoritmo es polinomial (n^k), y sabemos que un polinomio crece mucho más lento que una exponencial (2^n), entonces, cualquier cosa que termine en tiempo n^k definitivamente termina muchísimo antes de que se acabe el tiempo exponencial.

e. ¿Qué formula la Tesis Fuerte de Church-Turing?

La tesis fuerte de Church-Turing formula que, si un problema se resuelve en tiempo $\text{poly}(n)$ con un modelo computacional y una representación razonable (no una MTN, ni representación unaria, por ej), también lo hace otro modelo y representacion razonables.

Se puede pasar de máquinas y modelos equivalentes y seguro que no tardará más que $\text{poly}(n)$. Esto nos indica que podemos estudiar la complejidad intrínseca de un problema y abstraernos del modelo computacional y el sistema de representación.

(tesis puesta en duda por avances de la computación cuantica, pero por ahora sigue valiendo)

f. ¿Por qué es indistinta la cantidad de cintas de las MT que utilizamos para analizar los lenguajes, en el marco de la jerarquía temporal que definimos?

En el marco de la jerarquía temporal que definimos, la cantidad de cintas de las Máquinas de Turing (MT) que utilizamos es indistinta debido a que una MT con múltiples cintas no tiene mayor poder computacional que una MT con una sola cinta.

Cualquier tarea que pueda realizar una MT con K cintas, también puede ser realizada por una MT con una sola cinta.

Si una MT con K_1 cintas decide un lenguaje en tiempo polinomial, digamos $O(n^k)$ para alguna constante k , entonces existe una MT equivalente con K_2 cintas (incluso una sola cinta) que también decide el mismo lenguaje en tiempo polinomial.

Lo mismo se aplica para una resolución en tiempo exponencial.

g. ¿Qué codificación de cadenas se descarta en la complejidad temporal?

En cualquier base $k \geq 2$ (como binario o decimal), un número N se representa con $\log_2(N)$ símbolos. En unaria, ese mismo número requiere N símbolos.

Si un algoritmo tarda N pasos para procesar el número N :

- En binario, el tamaño de la entrada es $n = \log_2(N)$.
Por lo tanto, el tiempo es 2^n (Exponencial).
- En unaria, el tamaño de la entrada es $n = N$.
Por lo tanto, el tiempo parece $O(n)$ (Lineal).

Como la complejidad se calcula en función del tamaño de la entrada, usar unaria es como hacer trampa: hacés que la entrada sea artificialmente gigante para que al calcular la complejidad de un algoritmo lento, parezca que es rápido.

Por eso no se usa, tergiversa la verdadera complejidad de los lenguajes.

h. ¿Por qué si un lenguaje L pertenece a P , también su complemento L^c pertenece a P ?

Ayuda: hay que probar que si existe una MT M que decide L en tiempo $\text{poly}(n)$, también existe una MT M' que decide L^c en tiempo $\text{poly}(n)$.

Las máquinas de turing que usamos en P son deterministas y terminan siempre.

Si está en P , significa que tenemos una MT M que decide en tiempo $\text{poly}(n)$.

"Decidir" significa que para cualquier entrada, tarde o temprano, la máquina frena en el estado de Aceptación (q_A) o en el de Rechazo (q_R).

Entonces, podemos construir una MT M' que, ante una entrada w , hace lo siguiente:

- Ejecuta w sobre M
- Cuando M termina, M' permuta la respuesta (si M dijo q_A , M' dice q_R ; y si M dijo q_R , M' dice q_A)

M' tarda exactamente lo mismo que M , con solo un paso más que es invertir el resultado.

Ergo, sigue siendo $O(n^k)$ ($\text{poly}(n)$)

Como encontramos una máquina M' que decide L^c en tiempo polinomial, entonces $L^c \in P$. Esto se conoce técnicamente como que la clase P es cerrada bajo el complemento

i. Sea L un lenguaje de NP. Explicar por qué los certificados de L miden un tamaño polinomial con respecto al tamaño de las cadenas de entrada.

La definición de NP exige se puede **verificar** en tiempo $\text{poly}(n)$.

Si el certificado fuera exponencial con respecto a la entrada, nos tomaría tiempo exponencial el solo hecho de leer el certificado símbolo a símbolo, ergo ya no está en NP porque no lo estamos verificando en tiempo $\text{poly}(n)$.

Ejercicio 2. Sea el lenguaje $\text{SMALL-SAT} = \{\varphi \mid \varphi \text{ es una fórmula booleana sin cuantificadores en la forma normal conjuntiva (o FNC), y existe una asignación de valores de verdad que la satisface en la que hay a lo sumo 3 variables con valor de verdad verdadero}\}$. Probar que $\text{SMALL-SAT} \in \text{P}$.

Comentario: las fórmulas φ en la forma FNC son conjunciones de disyunciones de variables o variables negadas.

Ayuda: una MT que decide SMALL-SAT debe contemplar asignaciones con cero, uno, dos y hasta tres valores de verdad verdadero

Normalmente, en SAT, para saber si una fórmula es satisfacible, tenés que probar todas las combinaciones posibles de las variables. Si tenés n variables, tenés 2^n combinaciones (n está en el exponente, es $\exp(n)$).

Pero en SMALL-SAT tenemos, A LO SUMO 3 variables en verdadero, todas las demás deben estar en falso. Entonces, tenemos muchas menos combinaciones que probar, solo son casos donde hay poquitos true.

- Caso 0: Todas las variables en Falso (1 sola combinación).
- Caso 1: Solo una variable en Verdadero (n combinaciones posibles).
- Caso 2: Exactamente dos variables en Verdadero ($(n^2 / 2)$ combinaciones posibles).
- Caso 3: Exactamente tres variables en Verdadero ($(n^3 / 6)$ combinaciones posibles).

Para probar que algo está en P, tenemos que demostrar que existe un algoritmo que tarde un tiempo polinomial ($O(n^k)$).

Idea general:

- 1- Generar todas las asignaciones posibles que tengan 0, 1, 2 o 3 variables en T
- 2- Para cada una de esas asignaciones, evaluar si la formula da T
- 3- Si alguna da verdadero, qA. Si probamos todas y ninguna funcionó, qR.

Entonces el tiempo:

1- ¿cuántas asignaciones probamos?
La suma es $= 1 + n + (n^2 / 2) + (n^3 / 6) \rightarrow O(n^3)$

2- Evaluar una fórmula para una asignación dada es tiempo lineal respecto al tamaño de la fórmula $\rightarrow O(|\varphi|)$

Entonces, el tiempo es $O(n^3) \rightarrow \text{Polinomial} \rightarrow \text{SMALL-SAT} \in \text{P}$

Ejercicio 3. Dados los dos lenguajes siguientes,

(1) justificar por qué no estarían en P,

(2) probar que están en NP,

(3) justificar por qué sus complementos no estarían en NP:

a. El problema del conjunto dominante de un grafo consiste en determinar si un grafo no dirigido tiene un conjunto dominante de vértices. Un subconjunto D de vértices de un grafo G es un conjunto dominante de G , sii todo vértice de G fuera de D es adyacente a algún vértice de D . El lenguaje que representa el problema es $\text{DOM-SET} = \{(G, K) \mid G \text{ es un grafo no dirigido y tiene un conjunto dominante de } K \text{ vértices}\}$.

(1) Al igual que en CLIQUE, la forma conocida para decidir si existe un conjunto dominante de tamaño K es probar **todos los subconjuntos posibles** de K vértices (fuerza bruta).

- Si el grafo tiene n vértices, existen $C(n, k)$ combinaciones.
- Esto da un tiempo de ejecución de $O(n^K)$, lo cual es exponencial. Por lo tanto, no hay un algoritmo determinista conocido que lo resuelva en tiempo polinomial.
- **DOM-SET no estaría en P.**

(2) Para probar que está en NP, necesitamos un certificado y un proceso de chequeo $\text{poly}(n)$.

- **Certificado (C):** Un subconjunto de K vértices del grafo G .
- **Verificador:** Para cada vértice v del grafo que no esté en el certificado, nos fijamos si tiene al menos un vecino dentro del certificado C .
- **Tiempo:** Como máximo, recorreremos todos los vértices (V) y chequeamos sus aristas (E). Esto es $O(V+E)$ o $O(V^2)$, lo cual es claramente polinomial.
- **Como existe un certificado de tamaño polinomial y un verificador en tiempo polinomial, $\text{DOM-SET} \in \text{NP}$**

(3) El lenguaje complemento sería: $\text{DOM-SET}^c = \{(G, K) \mid G \text{ es un grafo que NO tiene un conjunto dominante de tamaño } k\}$

- Para verificar que el grafo SÍ tiene el conjunto dominante, el certificado solo tiene los K vertices. (punto anterior, el 2)
- Peeeero para verificar que NO EXISTE NINGUNO, tendríamos que mandar un certificado de que todos los posibles subconjuntos de tamaño K fallan.
- Como hay una cant exponencial de subconjuntos, ese certificado sería exponencialmente largo. Al no tener un certificado polinomial, el problema del complemento no puede estar en NP.

b. El problema de los grafos isomorfos consiste en determinar si dos grafos son isomorfos. Dos grafos son isomorfos si son idénticos salvo por la denominación de sus arcos. P.ej., dado el grafo $G1 = (\{1, 2, 3, 4\}, \{(1, 2), (2, 3), (3, 4), (4, 1)\})$, el grafo $G2 = (\{1, 2, 3, 4\}, \{(1, 2), (2, 4), (4, 3), (3, 1)\})$ es isomorfo a $G1$. El lenguaje que representa el problema de los grafos isomorfos es $ISO = \{(G1, G2) \mid G1 \text{ y } G2 \text{ son grafos isomorfos}\}$.

(1) Aunque no se ha demostrado que sea imposible, no se conoce ningún algoritmo determinista que resuelva el isomorfismo de grafos en tiempo polinomial para el caso general.

- La estrategia de "fuerza bruta" consistiría en probar todas las formas posibles de renombrar los nodos del grafo $G1$ para ver si queda igual a $G2$.
- Si los grafos tienen n nodos, hay $n!$ permutaciones posibles.
- El crecimiento de $n!$ es incluso más rápido que el de las exponenciales comunes (2^n). Por eso, para grafos grandes, es computacionalmente inviable.
- ISO no estaría en NP

(2) Para probar que está en NP , necesitamos un certificado y un proceso de chequeo $poly(n)$.

- **Certificado:** Una función de mapeo (una lista que diga "el nodo 1 de $G1$ es el nodo 3 de $G2$, el 2 es el 4...", etc.).
- **Verificador:** Se recibe esa lista y se chequea arista por arista: "Si en $G1$ el 1 está conectado con el 2, ¿en $G2$ el 3 está conectado con el 4?", etc.
- **Tiempo:** Chequear todas las aristas del grafo toma tiempo $O(n^2) \rightarrow poly(n)$.
- **Como se puede verificar en tiempo $poly(n)$, $ISO \in NP$**

(3) El complemento sería el lenguaje de los Grafos NO Isomorfos (digamos, NOT-ISO).

- Si dos grafos no son iguales, ¿qué "pista" cortita podría darte Merlín para convencerte rápido? No existe un "ejemplo" de que no son iguales. Habría que demostrar que ninguna de las $n!$ combinaciones funciona.
- Al igual que con el CLIQUE, esto requeriría una prueba de tamaño exponencial. No hay un certificado polinomial que se pueda chequear en su tiempo $poly(n)$ para estar seguro de que dos grafos son distintos

Ejercicio 4. Se prueba que NP = EXP. La prueba es la siguiente.

Si $L \in \text{NP}$, entonces existe una MT M que, para toda cadena de entrada w , verifica en tiempo $\text{poly}(|w|)$ si $w \in L$, con la ayuda de un certificado x tal que $|x| \leq p(|w|)$ - p es un polinomio -, y de esta manera, se puede construir una MT M' que decida en tiempo $\text{exp}(|w|)$ si $w \in L$, sin usar ninguna cadena adicional:

M' simplemente barre todos y cada uno de los certificados posibles x de w .

Se pide explicar por qué M' efectivamente tarda tiempo $\text{exp}(|w|)$.

Ayuda: como $|x| \leq p(|w|)$ y los símbolos de x pertenecen a un alfabeto de k símbolos, ¿cuántos certificados x de tamaño $p(|w|)$ tiene a lo sumo una cadena w ?

- ¿Cuántos certificados posibles existen?
 - Para longitud 1, hay k^1 combinaciones.
 - Para longitud 2, hay k^2 combinaciones.
 - Para la longitud máxima $p(n)$, hay $k^{p(n)}$ combinaciones
- Entonces, M' hace:
 - Genera c/u de los certificados
 - Corre el verificador (que nos dijeron que es polinomial) para CADA UNO de los certificados
 - **Ergo, el tiempo:**

$$\begin{aligned} T(n) &= (\text{cantCertificados}) \times (\text{tiempoVerificacionUnCertificado}) \\ &= k^{p(n)} \times \text{poly}(n) \\ &= O(\text{exp}(n)) \end{aligned}$$